PTM Architecture and Data Model Blueprint

Path to Marriage (PTM) | Version 2 | 11 October 2026

1. Decisions locked in

- Four account types: Standard, Premium, Assisted Premium, VIP
- Polyrepo: `ptm-backend`, `ptm-frontend`, `ptm-dashboard`
- Stack: Node.js, Express, MongoDB (Mongoose), TypeScript, Next.js, Tailwind CSS, Redis
- Standard accounts see basic info of others only: no chat, no phone, no contact. They get an immediate prompt to upgrade to Premium
- Guardian: one per member, a separate login linked to the member's user ID, with the same access as the member except chat
- Coupons, vouchers and discounts work like an ecommerce promotion system: start and end date and time, usage limits, one per checkout. This includes free-access coupons, such as 3 months of Premium for seminar attendees
- Payments: SSLCommerz (cards, bKash, Nagad) first; Stripe once the client provides API keys
- KYC rejection: an email asks for the correct information; if the user is still not satisfied, the refund goes to the original payment source
- Subscriptions follow the standard system (fixed-length plans, renewal, expiry), with plans managed from the dashboard
- Admin has full power over reports, SEO, marketing, finance, forecasting and every other area. Staff are limited to sales and marketing reports and KYC approval by default, and admin can give any staff member extra access, permanent or time-limited
- Screens in the design with no logic in the sheet are built to the design, and their logic is left for a later plan
- Chat: text only
- API documented with Swagger (OpenAPI), generated from the code itself
- Google and LinkedIn sign-up for faster registration. Members with an unfinished registration get reminders and are sent back to their registration step
- Member data is kept when an account is deactivated, so members can return, and a separate consented contact list serves marketing. Permanent erasure is available on request (section 7.7)
- Pricing: SSLCommerz charges in BDT and Stripe charges in GBP. Admin sets each plan's price in both currencies, with or without a discount (section 7.8)

2. Architecture at a glance

Style: modular monolith. One deployable API with strict module boundaries, not microservices. Payments, entitlements and KYC need strong consistency, and a small team ships faster with one codebase. Because modules talk only through interfaces and events, chat or payments can be extracted later without a rewrite.

Runtime components

- `ptm-frontend` (Next.js) calls the REST API at `/api/v1`
- `ptm-dashboard` (Next.js, separate domain) calls the same API under `/api/v1/admin` with staff tokens
- `ptm-backend`: API process (Express), worker process (BullMQ jobs), realtime (Socket.IO)
- MongoDB as a replica set (needed for transactions), Redis (cache, queues, rate limits, Socket.IO adapter), S3-compatible private storage (photos and KYC documents), email provider
- Three environments: dev, staging, production, each with its own database and secrets

Scaling path: stateless API containers behind a load balancer, Socket.IO with the Redis adapter, a separate worker process, cursor pagination everywhere, indexes designed from the search filters, reference data cached in Redis. Sharding is a later option, not a day-one need.

3. Backend structure

```
ptm-backend/src
  config/          env validation (Zod), db, redis, storage
  modules/
    auth/ users/ profiles/ signup/ kyc/ search/ connections/
    access/ chat/ notifications/ plans/ subscriptions/
    payments/ vouchers/ guardian/ assisted/ support/
    staff/ admin/ cms/ audit/
      *.routes.ts  *.controller.ts  *.service.ts
      *.repository.ts  *.model.ts  *.schema.ts  *.test.ts
  policies/        who can see or do what (single source of truth)
  providers/       payment, storage, mailer, sms, photo-verifier adapters
  events/          domain event bus and handlers
  jobs/            BullMQ workers (expiry, reconciliation, emails)
  middleware/      auth, role, permission, rate limit, error handler
  app.ts  server.ts  worker.ts
```

Layering rule: route, then controller (HTTP only), then service (business rules), then repository (Mongoose). Controllers never touch models. Services never touch req or res. A module never reads another module's models; it calls that module's service or listens to its events.

Cross-cutting rules

- Env variables validated at startup; the app refuses to boot if one is missing
- Every request body, query and param validated with Zod
- One error class with a code, and one error response shape for the whole API
- Structured logs (pino) with a request id on every line
- Money stored as integer minor units with a currency code, never floats

Domain events (in-process now, queued through BullMQ for slow work)

- `payment.succeeded` leads to the subscription being created
- `kyc.approved` leads to the badge change and a notification
- `request.accepted` leads to a conversation being created
- `subscription.expired` leads to the downgrade job
- `voucher.redeemed` leads to an audit entry

Adapters (swappable implementations behind interfaces)

- `PaymentProvider`: `SSLCommerzProvider`, `StripeProvider`. Adding a gateway means adding one class. SSLCommerz covers cards, bKash and Nagad. Stripe sits behind a feature flag until the client provides its API keys
- `StorageProvider`, `Mailer`, `SmsProvider`
- `PhotoVerifier`: manual staff check first, automated check later without touching callers

4. Access control

Identities: `member`, `guardian`, `staff`, `admin`, `super_admin`.

Tier is derived, not edited. A member's tier (`standard`, `premium`, `assisted_premium`, `vip`) comes from the active subscription plus KYC status. Nobody sets it by hand, which removes a whole class of bugs.

Policy layer. All rules live in `policies/` as plain functions: `canViewField`, `canSendRequest`, `canChat`, `canMessageFirst`, `canHideProfile`, `canDownloadBiodata`. Controllers call policies. The profile serializer applies `canViewField` to every field on the server, so blurred data never leaves the API; the frontend only renders a locked placeholder.

Field classes

- `public_basic` : visible to any logged-in member
- `standard_visible` : Standard viewers see basic info only (the fields on the Standard profile page: photo, age, height, marital status, religion, profession, education, nationality, residence, ethnicity, hometown, horoscope). No phone, email, address, family names or chat
- `sensitive` : needs an access grant from the owner. Email, phone, present and permanent address, office address, employer details, parents', siblings' and children's names, and the biodata PDF

Draft permission matrix

Capability	Standard	Premium	Assisted Premium	VIP
KYC required	No	Yes	Yes	Yes
Profile photos	Up to 3	Up to 7	Up to 7	Up to 7
Send and receive requests	Yes	Yes	Yes	Yes
View others' full profile	Basic info only	Yes, sensitive fields need owner grant	Same as Premium	Same as Premium
Read and send chat messages	No chat; upgrade prompt	Yes, after accepted request	Same as Premium	Yes, and can message anyone from search first
Hide from search	No	No	No	Yes
Crown badge	No	No	No	Yes
Download biodata PDF	No	With owner grant	With owner grant	With owner grant

Assisted Premium is assumed to behave like Premium once registered. The logic sheet only defines its application form.

Guardian rules. A guardian is a separate login linked to exactly one member's user ID, for example the parents of a Premium member. The guardian has the same access as the member, except the chat box: chat endpoints reject the guardian role in middleware, not

only in the UI. Assumed, to confirm: a guardian cannot change the member's password or email, manage billing, or delete the account. Every guardian action is written to an activity log the member can read, and the member can revoke the link at any time.

Staff and admin. Admin has full access and can view and amend every record. Staff have limited access by default: sales and marketing reports and KYC approval. Admin can extend it, as described next. Permissions are strings assigned per staff member, so more can be added later: `kyc.review`, `reports.sales`, `reports.marketing`, `users.suspend`, `assisted.register`, `support.reply`, `vouchers.manage`, `payments.view`, `payments.refund`, `plans.manage`, `staff.manage`, `cms.edit`, `audit.view`. Every sensitive action writes an append-only audit log entry.

Staff accounts and extra access

- Each staff member has a display name and a designation, for example KYC Reviewer or Marketing Executive. Staff edit their own name, designation and photo; admin can edit any staff account. A KYC decision records the reviewer's name and designation
- Default staff access is sales and marketing reports plus KYC approval. Everything else is refused by the API, not merely hidden from the menu
- Admin can grant extra permissions to any staff member at any time, either permanently or for a set period with an end date and time, with a note on why. Time-limited grants switch off automatically, and the staff member sees which extra access they hold and until when
- A permission is a module plus an action (view, create, edit, approve, export), so admin can allow, for example, view-only finance or edit access to a single CMS page
- For one-off work, such as a KYC backlog or a seminar voucher batch, admin grants the access the task needs and withdraws it when the work is done
- Every grant, change and withdrawal is written to the audit log with the admin who made it

5. Data model (MongoDB)

- **users**: email (unique), passwordHash (empty for sign-in-only accounts), authProviders (provider, providerUserId), role, status, emailVerifiedAt, lastLoginAt, registrationStatus, chosenPlan, lastStep, reminderCount, lastReminderAt
- **profiles**: userId (unique), name, dob, gender, maritalStatus, religion (+ other text), nationality, ethnicity, heightCm, bloodGroup, hometown, residingIn, residencyStatus, profession, educationLevel, contact details, addresses, jobs[], education[], family (father, mother, siblings[], children[]), assets[], photos[], isHidden, lastSearchFilters. Derived and stored: age, horoscope. Height is stored in cm so range queries are simple; feet and inches are a display concern

- **signup_drafts**: userId, flow, step, data, updatedAt (TTL index). Replaces browser-cache progress so a paid user never loses a half-finished signup
- **media**: ownerId, kind (profile_photo, id_front, id_back), storageKey, mime, size, realPhotoStatus. KYC documents live in a separate private bucket
- **kyc_submissions**: userId, tierRequested, idType, front and back media, status, reviewerId, reason, history[]
- **plans**: key, tier, durationDays, versioned prices per currency (BDT, GBP), each with amountMinor and an optional discount (type, value, label, startsAt, endsAt), active
- **subscriptions**: userId, planId, tier, status, startsAt, endsAt, source (payment, voucher, staff_grant), paymentId, redemptionId
- **payments**: userId, provider, providerRef, amountMinor, currency, status, price snapshot (list amount, plan discount, voucher discount, final amount, plan price version), voucherId, idempotencyKey (unique), sanitised webhook payloads
- **vouchers**: code (unique), reason (campaign name), type (`percent`, `fixed`, `free_access`), value, freeAccess {tier, durationDays}, startsAt and endsAt (date and time, stored in UTC), maxRedemptions, redeemedCount, perUserLimit, minOrderAmount, maxDiscountAmount, applicablePlans, status, createdBy
- **voucher_redemptions**: voucherId, userId, subscriptionId or paymentId, redeemedAt. Unique on (voucherId, userId)
- **connections**: fromUserId, toUserId, status, initiatedBy (member, guardian, vip_message), actedByGuardianId. Unique on the user pair
- **access_grants**: ownerId, viewerId, scope, status (requested, granted, revoked). Unique on (ownerId, viewerId)
- **exclusions**: pairs hidden from each other after Delete chat
- **conversations**: type (member, support), participants, connectionId, state, lastMessageAt
- **messages**: conversationId, senderId, text, createdAt, readBy. No attachment fields exist, and the API rejects unknown fields
- **support_threads**: guestSessionId or userId, status, assignedStaffId. Created only when the first message is sent
- **notifications**: userId, type, payload, readAt
- **guardian_links**: memberId (unique), guardianUserId (unique), status. The two unique indexes enforce 1 to 1
- **activity_logs**: memberId, actorId, actorRole, action, target, at
- **assisted_applications**: name, email, whatsapp, country, preferred time window, paymentId, status, assignedStaffId, createdUserId
- **audit_logs**: actorId, action, entityType, entityId, meta, ip, at (append-only)

- **staff_members**, **cms_pages**, **newsletter_subscribers**, **reference_data** (countries, cities, professions, education levels, religions; seeded and cached)

Search indexes. A compound index on the filter fields (`isHidden` , `tier` , `maritalStatus` , `religion` , `nationality` , `residingIn.country` , `heightCm` , `age`) plus a lowercase name prefix index for name search. Results use cursor pagination.

Staff, reporting and marketing collections

- **staff_members**: userId, displayName, designation, photo, basePermissions, status
- **permission_grants**: staffId, permissions, reason, grantedBy, startsAt, endsAt (empty means permanent), revokedAt
- **daily_metrics**: date, signups by tier, active members, KYC counts and turnaround, requests, messages, revenue by gateway, currency and plan, refunds, voucher discounts. A nightly job builds it so reports never scan live member collections
- **forecasts**: metric, method, assumptions, generatedAt, horizon, projected values (low, expected, high)
- **seo_settings** and **redirects**: per-page title, description, canonical, social preview, robots, structured data; old path, new path, status code
- **campaigns**: name, channel, UTM fields, start and end, linked vouchers, results
- **email_templates**: key, subject, body
- **archived_members**: memberId, encrypted profile snapshot, archivedAt, restoreUntil. Read only with the `members.restore` permission, and every read is audit logged
- **consents**: userId or contactId, type (terms, marketing email, marketing SMS), policy version, givenAt, withdrawnAt, source, ip
- **marketing_contacts**: email, name, country, plan interest, signup source, consent status. Never holds religion, ethnicity, blood group, family, assets, ID files, photos or chats
- **suppression_list**: hashed email, reason (unsubscribed, erased), addedAt. Keeps removed people from being mailed again
- **data_requests**: userId, type (export, erasure), status, receivedAt, dueAt, handledBy
- **blocklist**: hashed phone and hashed ID number of banned or fraudulent members, so they cannot simply re-register

6. State machines

- **KYC:** submitted, under_review, then approved, rejected or resubmit_requested (which returns to submitted). A rejection or resubmit request sends the user an email listing what to correct. The profile badge reads Pending approval until approved

- **Payment:** created, pending, then succeeded or failed; succeeded can become refunded, always to the original payment source
- **Subscription:** pending_signup, pending_kyc, active, expired (or cancelled)
- **Connection:** pending, then accepted or declined; accepted can become unmatched; unmatched can return to pending (Add again) or become deleted (final)
- **Guardian link:** invited, active, revoked
- **Voucher:** draft, active, then expired or disabled

Each machine is one function with an explicit allowed-transitions table, and each is unit tested for illegal transitions.

7. Key flows

7.1 Paid signup (Premium, VIP, Assisted Premium)

1. Visitor picks a plan and optionally applies one voucher
2. Backend creates a payment with an idempotency key and returns the SSLCommerz or Stripe redirect
3. The provider notifies the backend (SSLCommerz IPN plus validation call, or Stripe signed webhook). The handler verifies it and is idempotent
4. `payment.succeeded` creates the subscription as pending_signup; the user lands on signup page 1, or on the assisted application page
5. The 7-step signup is saved as a server draft after every step. Submit creates the profile and the KYC submission
6. Staff or admin approve in the dashboard, the subscription becomes active and the badge changes. If the documents are rejected, the user gets an email asking for the correct information and can resubmit. If the user is still not satisfied, the payment is refunded to its original source

A nightly reconciliation job compares provider records with local payments.

7.2 Coupons, vouchers and discounts

Coupons, vouchers and discounts follow the ecommerce pattern: a code the customer enters, or an automatic discount applied without a code.

- Types: percentage off, fixed amount off, or free access (a tier for a set number of days)
- Each has a reason or campaign name, a start and end date and time, a total usage limit, a per-user limit, an optional minimum order amount, an optional maximum discount, and the plans it applies to
- One voucher or coupon per checkout; they do not stack

- The server computes the price in a fixed order: plan price in the gateway's currency, then the plan discount if one is active, then the coupon or voucher, then the payable total. The browser never sends a price
- Redemption is atomic: one conditional update increments `redeemedCount` only if the voucher is active, inside its window and under its limit, with the unique (voucherId, userId) index inside a transaction
- Admin creates, edits and disables them, and every change is audit logged

Seminar example: a `free_access` voucher for Premium, 90 days. The total is zero, so no gateway call is made. The backend creates a subscription with source voucher and an end date of start plus 90 days. When the free period ends, the account returns to Standard unless the user pays.

7.3 Landing filters through login and signup

Submit stores the filters in a signed guest cookie. After login or signup they are saved to `lastSearchFilters` and applied to the first search. Remember-me users land directly on results with their saved filters.

7.4 Requests and chat

Any member can send a request. Accepting creates the conversation. Standard members cannot chat or see contact details; the API masks message text, and as soon as a Standard member tries to connect they get an immediate notification to upgrade to Premium. A VIP can message first from search; both sides then see Send request and Unmatch, and profile access opens only after both accept. Unmatch revokes access grants in both directions and blocks chat. Delete chat adds an exclusion so neither appears in the other's search.

7.5 Guardian

The member invites a guardian by email. The guardian accepts and receives a limited session. Their actions create activity log entries visible to the member. Chat endpoints reject the guardian role.

7.6 Google and LinkedIn sign-up, and unfinished registrations

The backend owns social sign-in, so the member site, the dashboard and any future app share one auth system.

1. The member taps Continue with Google or Continue with LinkedIn on the login page or on registration page 1. The plan they chose on the Pricing page, if any, travels with the request
2. The backend runs the authorization code flow with PKCE, state and nonce, and exchanges the code on the server. It keeps only the provider's user id, verified email, name and picture link

3. A known provider id logs the member in. An unknown one with an unused email creates an account with `registrationStatus` set to in_progress. An email already used by a password account must be confirmed with that password once before linking, and a provider email not marked verified is never trusted
4. Redirect by chosen plan: Standard goes to Standard signup, prefilled. Premium, VIP and Assisted Premium go to payment first, then registration page 1, prefilled. No plan chosen goes to the Pricing page, as the logic sheet says
5. Prefill covers first name, last name and email. Date of birth, gender and everything else are still entered by the member, and the under-18 rule still applies. Password fields are hidden for sign-in accounts. The provider photo is not used as a profile photo, because photos must be verified as real, and names must still match the ID for KYC

Unfinished registrations

While `registrationStatus` is not submitted, the router and the API send the member back to the step where they stopped (the API answers with a `registration_incomplete` error). Only registration, Pricing, Help and contact, legal pages and logout stay open. Reminders run from a scheduled BullMQ job:

When	What the member gets
Right after sign-in	A banner on every page and a redirect to their saved step
After 1 hour	Email with a one-tap link back to the saved step
Day 1, day 3, day 7	Email plus an in-app notification
Weekly, up to 8 weeks	Email, until registration is submitted
After 8 weeks	Emails stop and the account is marked dormant; the banner returns at next login

A member who has paid but not registered also appears in a follow-up list in the dashboard for sales staff. Step-by-step drop-off is recorded in `daily_metrics`, so reports show where members leave. Staff dashboard login stays password plus optional two-factor, with no social sign-in.

7.7 Account deletion, retention and marketing data

Members can always come back, and the business keeps a marketing audience, through three separate stores: live data, an encrypted archive, and a consented contact list.

1. **Two choices when a member deletes.** Deactivate (the default) removes the profile from search and from other members' chats, connections and access grants, and archives the data so the member can restore it by logging in or through support. Erase

my data permanently is carried out on request within the legal deadline: profile, photos, KYC files and the marketing contact are deleted or anonymised. Only what the law requires stays, such as payment records, plus a hashed email in the suppression list so the person is never mailed again. The deletion screen explains both choices in plain words

2. **Archive.** On deactivation, sensitive profile data moves to the encrypted `archived_members` store. Live collections hold no deactivated profile data, and other members see a Deleted member label in old chats
3. **Returning with a different email.** Never linked automatically, because a wrong match could hand one person another's profile. The member proves identity with a code sent to the phone number on file, or staff match their KYC details, and then staff with the `members.restore` permission restore the archive into the new account. The `blocklist` stops removed or banned members re-registering
4. **Sign-ups that never return**, for example through Google, stay as unfinished accounts with the reminders in 7.6
5. **Marketing contacts.** A separate `marketing_contacts` collection holds only contact basics, interest and consent status. Campaigns and segments read from it, never from member profiles. Signup shows an unticked marketing consent box, every email carries a one-click unsubscribe, and an unsubscribe adds the person to the suppression list

Retention schedule (proposed defaults, editable by admin in Settings, to be confirmed with a lawyer)

Data	Kept	Then
Deactivated profile and chats	24 months in the archive	Anonymise, or ask the member for consent to keep it
Unfinished sign-up	12 months after last activity	Delete
ID document images	90 days after the KYC decision	Delete the images; keep the result, document type and reviewer. A restored member uploads again
Payment and refund records	As accounting rules require, usually several years	Keep minimal fields
Marketing contact	Until consent is withdrawn, or 24 months without engagement	Delete; keep the hashed suppression entry
Audit log	Long term	Keep

A nightly `retention-sweep` job applies the schedule and logs what it did. The dashboard gets a Deactivated and archived view, a restore action, and a queue of data requests with due dates.

To confirm before launch: the retention periods above; what happens to an active paid plan when a member deactivates (pause or forfeit); and a privacy policy and terms that describe this retention. A lawyer should review them, because members live in several countries and more than one law applies.

7.8 Plan pricing, currencies and plan discounts

Every paid plan has its own price in two currencies, both set by admin: Taka (BDT) for SSLCommerz and pounds (GBP) for Stripe.

- **The gateway sets the currency.** SSLCommerz charges in BDT (cards, bKash and Nagad) and Stripe charges in GBP. A member can pay with a card in any currency, and their bank converts it. Checkout shows the exact amount that will be charged, in the chosen gateway's currency, before they pay
- **Admin enters each price directly,** not through a live exchange rate, so the amount shown is the amount charged and nothing shifts with the rate. The pricing page starts in BDT for visitors in Bangladesh and GBP elsewhere, and visitors can switch
- **Plan discount.** Admin can save a plan price with or without a discount: a percentage or a fixed amount, an optional label such as Launch offer, and optional start and end date and time. With a discount, the pricing page shows the list price crossed out beside the sale price. Without one, it shows the list price only
- **Price changes are versioned.** Editing a price creates a new version, optionally effective from a future date. Every payment keeps a snapshot of its amounts and the plan price version, so later edits never change what a member paid. Each change is audit logged with the admin's name
- **Order of calculation:** list price, then plan discount, then coupon or voucher, with one voucher per checkout. By default a voucher applies to the discounted price, and a voucher can be marked as not valid on already discounted plans
- **Rounding.** Amounts are integers in each currency's smallest unit, and one tested function does all rounding
- **Refunds** go back to the original payment source, in the currency and amount actually paid
- **Reporting.** Finance and reports show BDT and GBP separately. A combined total uses a reporting rate that admin sets, and the report labels it

Starting prices. These are the values seeded at launch; admin can edit any of them at any time.

Plan	Duration	BDT (SSLCommerz)	GBP (Stripe)	Status
Standard	None	Free	Free	Fixed
Premium	365 days	1,000	£10	Given by the client
Assisted Premium	365 days	2,000	£20	Placeholder
VIP	365 days	3,000	£30	Placeholder

To confirm: the placeholder prices for Assisted Premium and VIP, which admin edits before launch, and whether coupons should apply to discounted plans by default (assumed yes).

8. API conventions and Swagger

- Base path `/api/v1`, plural nouns, JSON, ISO dates
- One error shape: `{ error: { code, message, details, requestId } }`
- Cursor pagination: `{ items, nextCursor }`
- `Idempotency-Key` header on checkout and voucher redemption
- Auth: short-lived access token plus refresh token in an httpOnly secure cookie; the dashboard uses a separate token audience
- One Zod schema per endpoint produces validation, TypeScript types and the OpenAPI spec
- Swagger UI at `/api/docs` (restricted in production), `openapi.json` exported on every CI run
- Frontend and dashboard generate typed clients from `openapi.json`; CI fails if a backend change breaks a client

First endpoints

Group	Endpoints
Auth	register, login, oauth start and callback (google, linkedin), link provider, refresh, logout, forgot-password, reset-password
Signup	GET and PUT draft, POST submit
Profiles	GET and PATCH me, GET by id (policy filtered)
Search	filtered search, search by name

Group	Endpoints
Checkout	GET plans, validate voucher, POST checkout, webhooks for sslcommerz and stripe
KYC	submit, status; admin queue and decision
Connections	request, accept, decline, unmatch, add again, delete
Access	request access, grant, revoke
Chat	conversations, messages (REST) plus socket events
Guardian	invite, accept, revoke, activity log
Vouchers	admin create, list, edit, disable; member validates a code at checkout

9. Frontend and dashboard

ptm-frontend

- App Router with route groups: `(public)` landing, pricing, about, how-it-works, help; `(auth)` login and reset; `(registration)` standard 1 to 3, premium and VIP 1 to 7, assisted application; `(app)` search, profile, chat, notifications
- `features/` folder per domain with its components, hooks and API calls; `components/ui` is the design system built from the Figma tokens
- Server components for public SEO pages; client components for forms, search and chat
- TanStack Query for server data, React Hook Form with Zod for the wizard, Zustand only for small UI state
- `middleware.ts` guards routes by session and tier, but the API remains the real authority
- Mobile and desktop built together, since Figma has both. Height and age use increment and decrement steppers on the landing page, search and signup, with Any as the default on both (the logic sheet wins over the Figma defaults). Profile fields are edited and saved one input at a time on the profile page

ptm-dashboard (separate app and domain)

- Staff login with optional two-factor, short sessions, sidebar generated from the staff member's permissions

- Admin sees every module and can amend any record; staff see only sales and marketing reports and the KYC queue. The full module list is in the table below

Dashboard modules

Module	What admin controls	Staff by default
Overview	Live KPIs: signups, active members, revenue, KYC backlog, support load	Sales and marketing summary
Reports	Members, tiers, conversion funnel, retention, KYC turnaround, requests and chat activity; filters by date, country and tier; CSV export	Sales and marketing reports
Marketing	Vouchers and discounts, campaigns with UTM tracking, newsletter list and sends, seminar coupon batches, segments built from consented marketing contacts	View marketing reports
SEO	Per-page title, description, social preview, canonical, robots, sitemap, redirects, structured data	None
Finance	Payments by gateway, currency and plan, refunds, reconciliation, receipts, revenue, plan discount and voucher cost, exports; BDT and GBP shown separately, with any combined total using an admin-set reporting rate	None
Forecasting	Projected signups, revenue, renewals and expiries; scenario inputs such as growth and conversion; low, expected and high ranges	None
KYC	Full queue, assignment, approve or reject with reason, resubmission emails, history	Approve and reject
Members	Search, view, edit any field, suspend, change tier, restore deactivated members, handle data export and erasure requests	None
Assisted registrations	Applications, assignment, register on behalf	None
Support inbox	Guest and member chats, assignment	None

Module	What admin controls	Staff by default
Plans and pricing	Prices in BDT and GBP, optional plan discounts with dates, price history and scheduled changes, durations, renewal rules	None
CMS	About, Help and FAQ, policies, How it works, landing copy	None
Staff and access	Staff accounts, name and designation, base and extra access, time-limited grants	Own profile only
Audit log	Every sensitive action, filterable	None
Settings	Gateways, email and SMS templates, notification rules, feature flags	None

Reports read from `daily_metrics` (built by a nightly job, with a live view for today), so heavy reports never slow member traffic, and exports run as background jobs. Forecasts start simple, using trends from `daily_metrics` with low, expected and high ranges, and improve as history builds. In the first months they rest on little data, and the dashboard says so.

10. Quality and security gates

- TypeScript strict mode in all repos; ESLint, Prettier, Husky, lint-staged; conventional commits; protected main branch with required checks
- Tests: policy tests for every tier and capability, state machine tests, Supertest integration tests against a real MongoDB, recorded webhook payload tests, Playwright end-to-end for signup, checkout and chat
- CI: typecheck, lint, test, build, export OpenAPI, build clients, deploy to staging, manual approval to production
- Passwords hashed with argon2 or bcrypt; helmet; rate limits on login, chat and uploads; request sanitising; upload validation with image re-encoding and EXIF stripping
- Private buckets with short-lived signed URLs. ID documents encrypted at rest, and every view is audit logged
- Webhook signature verification; secrets in a secret manager, never in repos; dependency scanning

- Treat personal data law (UK GDPR style) as in scope: consent record at signup, retention and deletion rules for KYC documents. Confirm specifics with a lawyer; section 7.7 sets the retention design
- Sentry, health and readiness endpoints, uptime alerts, daily backups with a tested restore

11. Delivery milestones

- **M0 Foundations:** three repos, CI, Docker Compose (MongoDB replica set, Redis, local S3), env validation, error handling, OpenAPI pipeline, design tokens
- **M1 Auth and Standard signup:** register, login, Google and LinkedIn sign-in, registration gate and reminders, password reset, signup drafts, Standard profile, photo upload
- **M2 Search and connections:** filters, name search, cards, requests, access grants, field-level serializer
- **M3 Plans, vouchers, payments:** plans, checkout, SSLCommerz, Stripe, webhooks, free-access coupons, subscriptions
- **M4 Premium and VIP signup with KYC:** 7-step flow, ID upload, KYC queue in the dashboard, badges
- **M5 Chat and notifications:** Socket.IO text chat, Standard masking, VIP message-first, unmatch and delete, notifications
- **M6 Guardian and assisted registration:** guardian link and activity log, assisted application, staff register-on-behalf, account deactivation and restore, retention job
- **M7 Dashboard completion:** members, support inbox, voucher screens, finance, reports, SEO and marketing tools, forecasting, staff accounts with time-limited access, CMS, audit log
- **M8 Hardening and launch:** load test search and chat, security review, restore test, acceptance check against Figma

Definition of done for every module: OpenAPI docs, policy and state machine tests, seed and migration scripts, and a visual match to the Figma frame on desktop and mobile.

12. Open items and future plans

1. Final launch prices for Assisted Premium and VIP. Premium is set at 1,000 BDT or £10 per year, and the other two start as placeholders (section 7.8). Admin edits them in the dashboard
2. Does a free-access coupon still require KYC? (assumed yes)
3. Guardian limits: assumed no password or email change, billing, or account deletion

4. Future plan: Send Gifts, Assess Compatibility and the Shop page are in the design with no logic, so they are built to the design and their logic is planned later
5. Stripe stays switched off until the client provides API keys. Google and LinkedIn sign-in likewise need the client's app credentials and redirect URIs for staging and production
6. Real-photo check: manual staff review first (assumed), automated later

13. Full delivery plan

The build is about 30 weeks of work for one full-time developer, so plan for 34 to 36 weeks with buffer. Two developers, one on the backend and one on the frontend and dashboard, can finish in roughly 18 to 22 weeks. These are planning estimates that should be re-checked at the end of M1, once real speed is known.

Scope in one view: about 60 desktop and 55 mobile distinct Figma screens (around 200 frames in total), 4 account types, 7-step KYC registration, 2 payment gateways, Google and LinkedIn sign-in, real-time text chat, guardian access, vouchers and discounts, a staff and admin dashboard with reports, finance, SEO, marketing and forecasting, and a retention and deactivation system.

Milestone plan (one developer)

Milestone	Weeks	Ends at week	Main work by repo	Done when
M0 Foundations	1.5	1.5	Backend: scaffold, env validation, errors, logging, Docker Compose (MongoDB replica set, Redis, local S3), OpenAPI pipeline, CI. Frontend: Next.js scaffold, Tailwind tokens from Figma, typed API client, Storybook. Dashboard: scaffold and staff login shell	CI is green end to end, the typed client builds from `openapi.json`, and staging deploys

Milestone	Weeks	Ends at week	Main work by repo	Done when
M1 Auth and Standard signup	3	4.5	Backend: register, login, refresh, reset, Google and LinkedIn sign-in, signup drafts, photo upload, Standard profile, registration gate and reminder job. Frontend: landing page with steppers, header states, login, Pricing, Standard signup (3 steps), Standard profile	A new person signs up by email, Google or LinkedIn, finishes Standard registration and sees their profile; unfinished members are redirected and reminded
M2 Search and connections	3	7.5	Backend: search and name search, requests and unmatch rules, access grants, field-level serializer, in-app notifications. Frontend: results page and cards, filters, other members' profiles with blurred fields and upgrade prompts, notifications	Tier-by-capability tests pass, Standard sees basic info only, and landing filters survive login and signup

Milestone	Weeks	Ends at week	Main work by repo	Done when
M3 Plans, pricing and payments	4	11.5	Backend: versioned plan prices and discounts, checkout, SSLCommerz and Stripe providers, webhooks and reconciliation, subscriptions and expiry, vouchers with atomic redemption, refunds. Frontend: pricing page with currency switch, checkout, result pages. Dashboard: plans and pricing editor, vouchers, payments list	Sandbox payments pass success, failure, abandoned, duplicate-webhook and refund cases on both gateways, and a free-access voucher creates a 90-day subscription
M4 Premium, VIP and KYC	4	15.5	Backend: 7-step registration data, repeating groups, ID upload to private storage, KYC state machine, rejection emails, badges. Frontend: Premium and VIP flows (new and existing users), assisted application, Premium and VIP profiles, biodata PDF. Dashboard: staff accounts with name and designation, KYC queue	A paid member completes 7 steps, staff approve, the badge changes, and ID images are proven unreachable without a signed link

Milestone	Weeks	Ends at week	Main work by repo	Done when
M5 Chat and support	3	18.5	Backend: Socket.IO text chat, Standard masking, VIP message-first, mute, unmatch, delete, guest support threads. Frontend: chat pages, Contact us flow. Dashboard: support inbox	Two members chat in real time across two server instances, and staff see a guest's support chat only after the first message
M6 Guardian, assisted and retention	3	21.5	Backend: guardian link and activity log, register-on-behalf, deactivate, restore, erase, archive, retention job, consents, data requests. Frontend: guardian screens, account settings, consent boxes. Dashboard: assisted queue, archived members, data requests	A guardian can do everything except chat with every action logged; deactivate then restore works; erasure follows the schedule
M7 Dashboard completion	5	26.5	Backend: daily metrics, reports, forecasts, SEO settings and redirects, campaigns, time-limited permission grants, audit log API, CMS. Frontend: CMS-driven pages, SEO tags, sitemap. Dashboard: Overview, Reports, Marketing, SEO, Finance, Forecasting, Staff and access, CMS, Audit log, Settings	Every module in the dashboard table works, default staff cannot reach anything beyond their access (API tests), and a time-limited grant expires on schedule

Milestone	Weeks	Ends at week	Main work by repo	Done when
M8 Hardening and launch	3	29.5	All repos: load tests for search and chat, security review, backup restore drill, acceptance check against Figma, content and data seeding, soft launch	Launch checklist below is fully ticked

What the client or owner must supply (start in week 1, because approvals take time)

Item	Needed by	Why
Google and LinkedIn sign-in apps, with privacy policy and terms links	M1	Social sign-in cannot go live without them
Domain, DNS access, and an email sending domain set up for deliverability	M1	Reminders, KYC emails and receipts must reach inboxes
Managed MongoDB (replica set), Redis, private file storage, hosting	M0	Staging and production environments
SSLCommerz merchant account (sandbox now, live before launch)	M3	BDT payments, bKash and Nagad
Stripe account and keys (test keys now, live before launch)	M3	GBP payments
Prices for Assisted Premium and VIP, and the staff list	M3 and M4	Seed data and dashboard accounts
Decision on an SMS provider for phone verification codes	M6	Needed to restore members who return with a different email
Privacy policy, terms and retention review by a lawyer	Before M6	The retention design must match the published policy
Text for Help and FAQ, Policies and Compliance, About, How it works, and the YouTube video link	M7	CMS content for launch

Design plan

The Figma audit found gaps to close before the milestones that need them: Standard signup and Premium steps 1 to 7 have desktop frames only, Notifications has no mobile frame, some frames are named 2, 3 and 4, desktop and mobile numbering differ, and a few labels have typos (Natinality, Caucasion, Comapatibility). The dashboard has no design at all.

- Close the signup gaps before M1 (Standard) and M4 (Premium and VIP), and fix names and typos in the same pass
- Design the dashboard screens from the same design system during M3, so they are ready when M4 starts
- To match Figma exactly: extract tokens from Figma variables into the Tailwind config, build each component in Storybook first, and compare automated screenshots against Figma exports for every screen at 1440 and 390 pixel widths

Risks and how the plan handles them

Risk	Effect	Handling
Features appear mid-build with no logic written (gifts, compatibility, shop)	Delay	Scope frozen per milestone; new requests go on a change list with an estimate; these three are in the Phase 2 list
Approvals from Google, LinkedIn, SSLCommerz or Stripe arrive late	Blocks M1 or M3	Apply in week 1, develop against sandboxes, keep gateways behind feature flags
Leak of KYC documents or sensitive profile fields	Severe	Private storage, encryption, signed links, audit logs, field-level serializer, security review before launch
Tier visibility bugs	Data exposure	One policy layer with a test for every tier and capability
Payment edge cases (duplicate webhooks, abandoned checkouts, refunds)	Wrong charges	Idempotency keys, nightly reconciliation, full sandbox test matrix
Search slows as members grow	Poor experience	Indexes from the filter list, cursor pagination, load test with 100,000 seeded profiles

Risk	Effect	Handling
Design gaps or mismatches	Rework	Gaps closed before each milestone, screenshot comparison against Figma
One person carries the whole build	Delay if unavailable	Small pull requests, written docs, CI on every change; add a second developer for the dashboard stream if the timeline matters
Retention rules conflict with local law	Rework	Lawyer review before M6; periods are admin settings, not code

Ways of working

- One pull request per feature, merged only when checks pass, with staging deployed on every merge
- A short demo against Figma at the end of each milestone, and written sign-off from the client
- Changes after sign-off go on the change list with an estimate

Launch checklist

- [ ] Acceptance check of every screen against Figma at desktop and mobile widths
- [ ] Load tests for search and chat pass
- [ ] Security review done and findings fixed
- [ ] Backups running and a restore drill completed
- [ ] Plans, prices, reference data (countries, cities, professions) and CMS content seeded
- [ ] SSLCommerz and Stripe live, with a real low-value payment and refund tested on each
- [ ] Google and LinkedIn sign-in live
- [ ] Privacy policy and terms published and linked in signup
- [ ] Staff accounts created with the right access
- [ ] Monitoring, alerts and a written runbook in place, with a rollback plan
- [ ] Soft launch to a limited group before opening to everyone

Phase 2 backlog (designed or mentioned, logic not written)

- Send Gifts and Assess Compatibility (Premium features on the landing page)
- Shop (currently a coming-soon page)
- Automated real-photo verification, after manual review has run for a while
- Media in chat (images and voice), after text-only chat is stable
- Richer forecasting once there is enough history